# Descenso del Gradiente

Notas de clase basadas en el Cap. 7 de *Deep Learning: Foundations and Concepts*
(C. M. Bishop y H. Bishop, Springer, 2024)

Universidad Nacional de Ingeniería — FIIS, Unidad de Posgrado
Redes Neuronales y Aprendizaje Profundo (Clase 2)

**Sobre este documento**

Estas notas son una elaboración propia y resumida de los temas que cubre el Capítulo 7 del libro de Bishop & Bishop (superficies de error, descenso del gradiente y sus variantes, convergencia, y normalización). No reproducen el texto, las figuras ni los ejercicios del libro; para el desarrollo completo y las demostraciones se recomienda consultar el capítulo original (versión en línea gratuita en `https://www.bishopbook.com/`). La notación es la habitual en el curso: $\boldsymbol{w}$ denota el vector de todos los parámetros y $E(\boldsymbol{w})$ la función de error.

## Índice

**1. Superficies de error** **2**

1.1. Puntos estacionarios . . . 2
1.2. Aproximación cuadrática local . . . 2
1.3. Simetrías del espacio de pesos . . . 3

**2. Optimización por descenso del gradiente** **3**

2.1. Idea básica . . . 3
2.2. Descenso por lotes completos . . . 3
2.3. Descenso del gradiente estocástico (SGD) . . . 3
2.4. Mini-lotes . . . 3
2.5. Inicialización de parámetros . . . 4

**3. Convergencia** **4**

3.1. Análisis en el caso cuadrático . . . 4
3.2. Momento . . . 4
3.3. Programación de la tasa de aprendizaje . . . 5
3.4. RMSProp y Adam . . . 5

**4. Normalización** **6**

4.1. Normalización de datos . . . . . 6
4.2. Normalización por lotes (*batch normalization*) . . . . . 6
4.3. Normalización por capas (*layer normalization*) . . . . . 7

**5. Resumen y preguntas de repaso** **7**

# 1. Superficies de error

Entrenar una red neuronal consiste en resolver el problema de optimización

$$\boldsymbol{w}^\star = \arg\min_{\boldsymbol{w}\in\mathbb{R}^D} E(\boldsymbol{w}), \qquad E(\boldsymbol{w}) = \sum_{n=1}^{N} E_n(\boldsymbol{w}), \tag{1}$$

donde $E_n$ es el error asociado al $n$-ésimo ejemplo de entrenamiento. Para una red con millones de parámetros, $E(\boldsymbol{w})$ es una superficie en un espacio de dimensión $D$ extremadamente alta, no convexa y con muchas simetrías.

## 1.1. Puntos estacionarios

Un punto $\boldsymbol{w}^\star$ es *estacionario* si $\nabla E(\boldsymbol{w}^\star) = \mathbf{0}$. Un punto estacionario puede ser un mínimo (local o global), un máximo o un *punto silla*. Para distinguirlos se usa la matriz hessiana $\boldsymbol{H} = \nabla\nabla E$, con $H_{ij} = \partial^2 E/\partial w_i \partial w_j$:

- $\boldsymbol{H}$ definida positiva $\Rightarrow$ mínimo local;
- $\boldsymbol{H}$ definida negativa $\Rightarrow$ máximo local;
- $\boldsymbol{H}$ con autovalores de ambos signos $\Rightarrow$ punto silla.

En dimensión alta los puntos silla son mucho más frecuentes que los mínimos locales "malos", lo que explica en parte por qué el ruido del descenso estocástico resulta útil (Sección 2.3).

## 1.2. Aproximación cuadrática local

Alrededor de un mínimo $\boldsymbol{w}^\star$, el desarrollo de Taylor de segundo orden es

$$E(\boldsymbol{w}) \simeq E(\boldsymbol{w}^\star) + \tfrac{1}{2}(\boldsymbol{w}-\boldsymbol{w}^\star)^\top \boldsymbol{H}\,(\boldsymbol{w}-\boldsymbol{w}^\star). \tag{2}$$

Sea $\boldsymbol{H}\boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i$ la descomposición espectral (con $\boldsymbol{u}_i$ ortonormales) y escribamos $\boldsymbol{w}-\boldsymbol{w}^\star = \sum_i \alpha_i \boldsymbol{u}_i$. Entonces

$$E(\boldsymbol{w}) \simeq E(\boldsymbol{w}^\star) + \tfrac{1}{2}\sum_{i=1}^{D} \lambda_i\, \alpha_i^2. \tag{3}$$

Los contornos de error son elipsoides cuyos ejes apuntan según los autovectores; la curvatura a lo largo de $\boldsymbol{u}_i$ es $\lambda_i$. La razón entre el mayor y el menor autovalor,

$$\kappa = \frac{\lambda_{\text{máx}}}{\lambda_{\text{mín}}}, \tag{4}$$

es el *número de condición*: cuanto mayor es $\kappa$, más "alargado" es el valle y más difícil resulta optimizar.

### 1.3. Simetrías del espacio de pesos

Distintos valores de $\boldsymbol{w}$ pueden producir exactamente la misma función. En una capa oculta con $M$ unidades y activación tanh (impar), se pueden permutar las unidades ($M!$ formas) y cambiar el signo de todos los pesos entrantes y salientes de una unidad ($2^M$ formas), de modo que cada mínimo aparece repetido $M!\,2^M$ veces. Esto implica que existen muchísimos mínimos equivalentes y que romper la simetría en la inicialización es indispensable (Sección 2.5).

## 2. Optimización por descenso del gradiente

### 2.1. Idea básica

El gradiente $\nabla E(\boldsymbol{w})$ apunta en la dirección de máximo crecimiento local del error. Por tanto, moverse en sentido contrario reduce $E$ para pasos suficientemente pequeños:

$$\boxed{\boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} - \eta\,\nabla E(\boldsymbol{w}^{(\tau)})} \tag{5}$$

El hiperparámetro $\eta > 0$ es la *tasa de aprendizaje* (*learning rate*). El gradiente se calcula eficientemente mediante retropropagación (Capítulo 8 del libro; también parte de la Clase 2 del curso).

### 2.2. Descenso por lotes completos

Usar (5) con $\nabla E = \sum_n \nabla E_n$ se llama descenso por lotes (*batch gradient descent*). Cada actualización requiere recorrer todo el conjunto de datos, lo que es costoso si $N$ es grande, y el método puede quedar atrapado en regiones planas o puntos silla.

### 2.3. Descenso del gradiente estocástico (SGD)

Como $E$ es una suma sobre ejemplos, se puede actualizar los parámetros usando el gradiente de un solo ejemplo (o un subconjunto):

$$\boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} - \eta\,\nabla E_n(\boldsymbol{w}^{(\tau)}). \tag{6}$$

Este gradiente es una estimación ruidosa e insesgada del gradiente completo. Ventajas: cada paso es barato, se aprovecha la redundancia entre ejemplos, y el ruido ayuda a escapar de puntos silla y mínimos poco profundos. Es el método de la familia de Robbins y Monro (1951) para aproximación estocástica.

### 2.4. Mini-lotes

En la práctica se usan *mini-lotes* de $B$ ejemplos:

$$\boldsymbol{g}^{(\tau)} = \frac{1}{B} \sum_{n \in \mathcal{B}_\tau} \nabla E_n(\boldsymbol{w}^{(\tau)}). \tag{7}$$

Compromisos a recordar:

- La varianza del estimador del gradiente disminuye aproximadamente como $1/B$.

- Lotes mayores aprovechan mejor el paralelismo de GPU, pero con rendimientos decrecientes.
- Una *época* es un recorrido completo por los datos, normalmente con los datos barajados en cada época.

### 2.5. Inicialización de parámetros

Los pesos deben inicializarse aleatoriamente (para romper la simetría) y con una escala adecuada: si es demasiado grande o demasiado pequeña, las activaciones y los gradientes se saturan, explotan o se desvanecen al propagarse por muchas capas. Reglas estándar, con $n_{\text{ent}}$ y $n_{\text{sal}}$ el número de entradas y salidas de la capa:

| Esquema | Varianza de $w_{ij}$ | Uso típico |
|---|---|---|
| Xavier / Glorot | $\dfrac{2}{n_{\text{ent}} + n_{\text{sal}}}$ | tanh, sigmoide |
| He | $\dfrac{2}{n_{\text{ent}}}$ | ReLU |

La idea es mantener aproximadamente constante la varianza de las activaciones (y de los gradientes) de una capa a la siguiente. Los sesgos suelen iniciarse en cero.

## 3. Convergencia

### 3.1. Análisis en el caso cuadrático

Con la aproximación (3) el gradiente en la dirección $\boldsymbol{u}_i$ es $\lambda_i \alpha_i$, y la actualización (5) da

$$\alpha_i^{(\tau+1)} = (1 - \eta\lambda_i)\, \alpha_i^{(\tau)} \quad \Longrightarrow \quad \alpha_i^{(\tau)} = (1 - \eta\lambda_i)^{\tau} \alpha_i^{(0)}. \tag{8}$$

Por lo tanto:

- Convergen todas las componentes si y solo si $|1 - \eta\lambda_i| < 1$ para todo $i$, es decir,

$$0 < \eta < \frac{2}{\lambda_{\text{máx}}}. \tag{9}$$

- La componente con menor curvatura $\lambda_{\text{mín}}$ converge lentamente: su factor de contracción es $1 - \eta\lambda_{\text{mín}}$.
- Con $\eta = 2/(\lambda_{\text{máx}} + \lambda_{\text{mín}})$ (valor que minimiza el peor factor) la contracción por paso es

$$\frac{\kappa - 1}{\kappa + 1}, \tag{10}$$

que se acerca a 1 cuando $\kappa$ es grande: el método avanza a "zig-zag" a través del valle estrecho.

### 3.2. Momento

Para amortiguar las oscilaciones y acelerar el avance a lo largo de direcciones de curvatura baja se acumula una "velocidad":

$$\boldsymbol{v}^{(\tau)} = \mu\, \boldsymbol{v}^{(\tau-1)} - \eta\, \boldsymbol{g}^{(\tau)}, \qquad \boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} + \boldsymbol{v}^{(\tau)}, \qquad 0 \le \mu < 1. \tag{11}$$

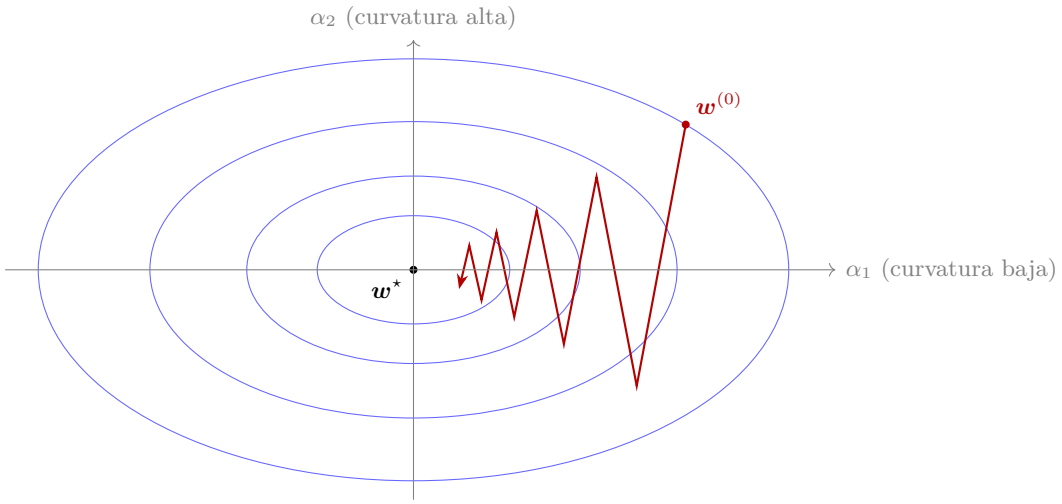



*Figura 1.* Zig-zag del descenso del gradiente sobre un valle mal condicionado ($\kappa = 10$, $\eta = 0{,}18$). Ilustración propia (escala vertical ampliada).

Si el gradiente es aproximadamente constante, la tasa efectiva es $\eta/(1-\mu)$: con $\mu = 0{,}9$, unas 10 veces mayor. Las componentes que oscilan de signo tienden a cancelarse. La variante de *Nesterov* evalúa el gradiente en el punto "adelantado" $\boldsymbol{w}^{(\tau)} + \mu \boldsymbol{v}^{(\tau-1)}$.

### 3.3. Programación de la tasa de aprendizaje

Con SGD, una tasa constante deja al algoritmo fluctuando alrededor del mínimo; conviene reducir $\eta$ a lo largo del entrenamiento. Las condiciones clásicas para convergencia estocástica son $\sum_\tau \eta_\tau = \infty$ y $\sum_\tau \eta_\tau^2 < \infty$. Programaciones habituales:

- **Decaimiento escalonado:** $\eta$ se multiplica por un factor $< 1$ cada cierto número de épocas.
- **Decaimiento exponencial:** $\eta_\tau = \eta_0\, e^{-k\tau}$.
- **Coseno:** $\eta_\tau = \frac{1}{2}\eta_0(1 + \cos(\pi\tau/T))$.
- **Calentamiento (*warm-up*):** aumento lineal al inicio, muy usado en transformadores.

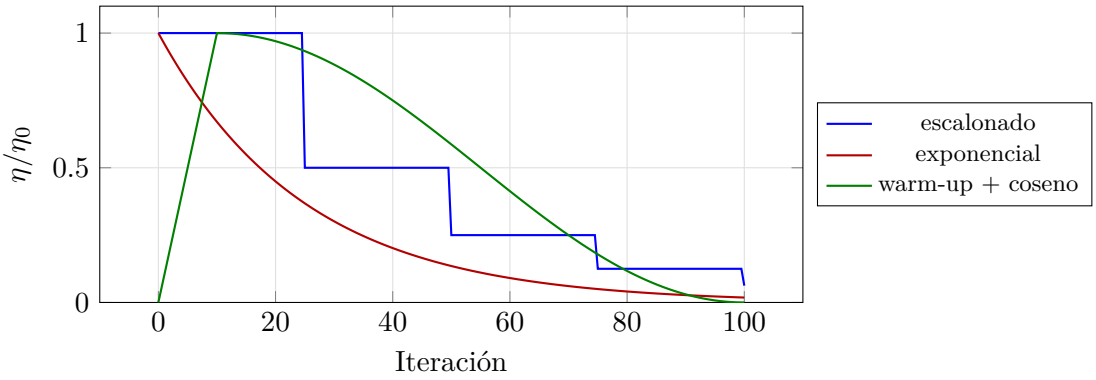



*Figura 2.* Ejemplos de programaciones de la tasa de aprendizaje. Ilustración propia.

### 3.4. RMSProp y Adam

Los métodos *adaptativos* usan una tasa distinta para cada parámetro, dividiendo por una estimación de la magnitud reciente de su gradiente. Con $g_i$ la $i$-ésima componente del gradiente

del mini-lote:

**RMSProp.**

$$r_i \leftarrow \beta\, r_i + (1-\beta)\, g_i^2, \qquad w_i \leftarrow w_i - \frac{\eta}{\sqrt{r_i}+\delta}\, g_i, \tag{12}$$

con $\beta \approx 0{,}9$ y $\delta \approx 10^{-8}$ para estabilidad numérica.

**Adam** (Kingma y Ba, 2015) combina momento y RMSProp, con corrección del sesgo inicial:

$$\boldsymbol{m} \leftarrow \beta_1 \boldsymbol{m} + (1-\beta_1)\boldsymbol{g}, \qquad \boldsymbol{s} \leftarrow \beta_2 \boldsymbol{s} + (1-\beta_2)\boldsymbol{g} \odot \boldsymbol{g}, \tag{13}$$

$$\hat{\boldsymbol{m}} = \frac{\boldsymbol{m}}{1-\beta_1^{\tau}}, \qquad \hat{\boldsymbol{s}} = \frac{\boldsymbol{s}}{1-\beta_2^{\tau}}, \tag{14}$$

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\, \frac{\hat{\boldsymbol{m}}}{\sqrt{\hat{\boldsymbol{s}}}+\delta}. \tag{15}$$

Valores por defecto habituales: $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\delta = 10^{-8}$. Las divisiones y raíces son componente a componente. La corrección del sesgo compensa que $\boldsymbol{m}$ y $\boldsymbol{s}$ se inicializan en cero y, por eso, subestiman las medias al principio.

**Resumen práctico**

SGD con momento y buena programación de $\eta$ sigue siendo competitivo (p. ej. en visión). Adam es una opción robusta por defecto y menos sensible a la escala de $\eta$; en transformadores se usa habitualmente junto con *warm-up* y decaimiento coseno.

# 4. Normalización

La normalización mejora el condicionamiento del problema y estabiliza el entrenamiento.

## 4.1. Normalización de datos

Cada característica de entrada se estandariza con la media y la desviación estándar calculadas *solo* sobre el conjunto de entrenamiento:

$$\tilde{x}_{ni} = \frac{x_{ni}-\mu_i}{\sigma_i}, \qquad \mu_i = \frac{1}{N}\sum_n x_{ni}, \quad \sigma_i^2 = \frac{1}{N}\sum_n (x_{ni}-\mu_i)^2. \tag{16}$$

Sin esto, características con escalas muy distintas generan valles alargados (números de condición grandes). Las mismas $\mu_i, \sigma_i$ deben aplicarse a validación y prueba.

## 4.2. Normalización por lotes (*batch normalization*)

Para las preactivaciones $z_{ni}$ de una capa oculta, con un mini-lote $\mathcal{B}$ de tamaño $B$:

$$\mu_i = \frac{1}{B}\sum_{n\in\mathcal{B}} z_{ni}, \quad \sigma_i^2 = \frac{1}{B}\sum_{n\in\mathcal{B}} (z_{ni}-\mu_i)^2, \quad \hat{z}_{ni} = \frac{z_{ni}-\mu_i}{\sqrt{\sigma_i^2+\delta}}, \quad \tilde{z}_{ni} = \gamma_i \hat{z}_{ni} + \beta_i. \tag{17}$$

Los parámetros $\gamma_i$ y $\beta_i$ son *aprendibles* y permiten a la red recuperar cualquier media y varianza que le resulten convenientes. En inferencia se sustituyen $\mu_i, \sigma_i^2$ por promedios móviles acumulados durante el entrenamiento. Sus efectos prácticos: permite tasas de aprendizaje mayores, reduce la sensibilidad a la inicialización y actúa como regularizador leve (por el ruido del mini-lote). Su desventaja: depende del tamaño del lote y se comporta distinto en entrenamiento e inferencia.

### 4.3. Normalización por capas (*layer normalization*)

Se calculan la media y la varianza sobre las *características de un mismo ejemplo* (no sobre el lote):

$$\mu_n = \frac{1}{M}\sum_{i=1}^{M} z_{ni}, \qquad \sigma_n^2 = \frac{1}{M}\sum_{i=1}^{M}(z_{ni}-\mu_n)^2, \qquad \hat{z}_{ni} = \frac{z_{ni}-\mu_n}{\sqrt{\sigma_n^2+\delta}}, \tag{18}$$

seguida de la misma transformación aprendible $\gamma_i \hat{z}_{ni} + \beta_i$. Es independiente del tamaño del lote y se comporta igual en entrenamiento e inferencia, por lo que es la elección estándar en redes recurrentes y transformadores.

| | Batch norm | Layer norm |
|---|---|---|
| Estadísticos sobre | el lote (por característica) | las características (por ejemplo) |
| Depende del tamaño del lote | sí | no |
| Difiere en inferencia | sí (promedios móviles) | no |
| Uso típico | CNN | Transformers, RNN |

## 5. Resumen y preguntas de repaso

**Ideas clave.**

1. El error de una red es una superficie no convexa con puntos silla y simetrías; localmente se aproxima por una cuadrática cuyo condicionamiento ($\kappa$) gobierna la velocidad de convergencia.
2. El descenso del gradiente estocástico con mini-lotes intercambia exactitud por eficiencia, y su ruido resulta beneficioso.
3. Momento, programación de $\eta$ y métodos adaptativos (RMSProp, Adam) aceleran y estabilizan la convergencia.
4. La inicialización adecuada y las normalizaciones (de datos, por lotes y por capas) hacen el problema mejor condicionado.

**Preguntas de repaso.**

1. Para $E(\boldsymbol{w}) = \frac{1}{2}(\lambda_1 w_1^2 + \lambda_2 w_2^2)$, ¿para qué valores de $\eta$ converge el descenso del gradiente? Calcule el $\eta$ óptimo y la contracción por paso para $\lambda_1 = 1$, $\lambda_2 = 10$.
2. ¿Por qué no se pueden inicializar todos los pesos en cero? ¿Qué cambia si solo los sesgos son cero?
3. Demuestre que el gradiente de un mini-lote uniformemente muestreado es un estimador insesgado del gradiente completo (escalado adecuadamente).
4. Con $\mu = 0{,}9$, ¿cuál es la tasa de aprendizaje efectiva del método de momento en una región de gradiente constante?
5. ¿Por qué Adam necesita la corrección del sesgo? Muestre qué valor toma $\boldsymbol{m}$ tras el primer paso sin corrección.
6. Compare batch norm y layer norm: ¿cuál usaría con lotes de tamaño 2, y por qué?

## Referencias

Bishop, C. M., & Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer. Cap. 7.

Robbins, H., & Monro, S. (1951). A stochastic approximation method. *Annals of Mathematical Statistics*.

Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*.

Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers. *ICCV*.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*.

Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv:1607.06450*.